

Analiza wydajności i granic stosowalności algorytmów: Dwumian Newtona oraz Algorytm Euklidesa

Kacper Korzekwa, gr II

15 kwietnia 2026

1 Wprowadzenie

Opierając się na programie z wykładu, zmierzyłem czas działania algorytmów i określiłem granice stosowalności w zależności od wartości parametrów. Ze względu na bardzo dużą szybkość obu algorytmów, pomiar czasu został wykonany dla bloku 1 000 000 wywołań dla każdej pary parametrów.

2 Pomiary wydajności i wyniki dla Dwumianu Newtona

Parametry (n, k)	Czas Iteracyjny [s]	Czas Rekurencyjny [s]	Wynik (Res)
(30, 15)	0.110259	0.129528	-131213633
(31, 15)	0.113448	0.136675	-119517046
(32, 15)	0.108056	0.130012	125213255
(33, 15)	0.137353	0.138484	-9051659
(34, 15)	0.111752	0.129557	-126325076
(35, 15)	0.112942	0.135850	-77903316
(36, 15)	0.114945	0.128508	64680748
(37, 15)	0.111410	0.135145	101058017
(38, 15)	0.113049	0.126732	109234020
(39, 15)	0.111750	0.138515	-1451685

Granice stosowalności: Na podstawie uzyskanych wyników z powyższej tabeli widać, że dla większości parametrów program zwraca wartości ujemne lub nielogiczne. Świadczy to o osiągnięciu głównej granicy stosowalności tej implementacji (*integer overflow*). Ponieważ funkcja operuje na 32-bitowym `int`, każda wartość przekraczająca zakres ok. 2,14 miliarda powoduje błędną interpretację bitu znaku. Ze względu na operacje mnożenia wewnątrz algorytmu, dla badanych wartości n oraz k program przestaje działać poprawnie, co sugeruje konieczność przejścia na typy o większej precyzji.

3 Pomiary wydajności i wyniki dla Algorytmu Euklidesa

Badane Liczby	Czas Iteracyjny [s]	Czas Rekurencyjny [s]	NWD (Res)
(2147483647, 1000000)	0.117904	0.147431	1
(2147483647, 1000001)	0.0705667	0.0828185	1
(2147483647, 1000002)	0.115997	0.140560	1
(2147483647, 1000003)	0.0905581	0.116821	1
(2147483647, 1000004)	0.199285	0.261718	1
(2147483647, 1000005)	0.111843	0.133688	1
(2147483647, 1000006)	0.0971659	0.175882	1
(2147483647, 1000007)	0.0746873	0.0688579	1
(2147483647, 1000008)	0.143699	0.161133	1
(2147483647, 1000009)	0.0901527	0.112762	1

Badane Liczby	Czas Iteracyjny [s]	Czas Rekurencyjny [s]	NWD (Res)
(2147483647, 1000010)	0.125070	0.120422	1
(2147483647, 1000011)	0.102175	0.128392	1
(2147483647, 1000012)	0.104982	0.115418	1
(2147483647, 1000013)	0.0818389	0.166404	1
(2147483647, 1000014)	0.123562	0.144090	1
(2147483647, 1000015)	0.146143	0.153428	1
(2147483647, 1000016)	0.111992	0.140741	1
(2147483647, 1000017)	0.0669487	0.0893213	1
(2147483647, 1000018)	0.109307	0.130451	1
(2147483647, 1000019)	0.123566	0.153072	1

Granice stosowalności: W przypadku algorytmu Euklidesa główną barierą jest zakres typu `int`. Wykorzystanie dużej liczby pierwszej ($2^{31} - 1$) pozwoliło sprawdzić stabilność algorytmu przy wartościach bliskich limitom sprzętowym. Algorytm jest na tyle wydajny czasowo, że nie napotyka barier przy typowych obliczeniach. Teoretycznym ograniczeniem wersji rekurencyjnej jest głębokość stosu, jednak logarytmiczna złożoność NWD sprawia, że błąd *stack overflow* jest nieosiągalny dla standardowych danych.

4 Podsumowanie

Przeprowadzone testy porównawcze wykazały, że wersje iteracyjne są średnio o 15-25% wydajniejsze od wersji rekurencyjnych, co widać przy pomiarach miliona wywołań. Dzieje się tak, ponieważ pętle nie tracą czasu na ciągłą obsługę wywołań funkcji i stosu. Głównym wnioskiem jest to, że w C++ częściej problemem jest brak miejsca w typie zmiennej na wynik (przepełnienie w dwumianie) niż sama szybkość procesora. Do większych obliczeń konieczna byłaby zmiana typu na `long long`.